

TJSC 2026 - Analista de Sistemas - Banca FGV

Arquitetura e Integração

Aula 1

MVC, cliente-servidor e arquitetura em camadas

Material autoral de revisão estratégica

Foco: converter experiência prática em acerto de prova

Versão: 27/04/2026

Contents

1 Como estudar esta aula

Esta aula foi desenhada para preparação objetiva para o TJSC 2026, cargo Analista de Sistemas, banca FGV. Você já trabalha com sistemas corporativos, APIs, Angular, TypeScript, Java/Quarkus e integrações. O objetivo aqui não é ensinar programação do zero, mas transformar essa vivência em resposta correta.

Bloco recomendado: 25 minutos para cliente-servidor; 25 minutos para camadas e MVC; 10 minutos para revisar os quadros; 35 a 45 minutos para resolver as 10 questões; 15 minutos para transformar erros em flashcards.

Meta da aula: diferenciar três ideias que a FGV pode misturar: arquitetura cliente-servidor, arquitetura em camadas e padrão MVC.

Resumo inicial:

- Cliente-servidor: **cliente solicita, servidor processa e responde.**
- Camadas: **organização lógica por responsabilidades**, como apresentação, aplicação/negócio e persistência.
- MVC: separação entre Model, View e Controller, principalmente na camada de apresentação/web.
- Tier: **separação física de implantação**, como **máquinas, processos, clusters ou servidores.**

2 Enquadramento no edital TJSC/FGV

No edital de Analista de Sistemas, a disciplina Arquitetura de Sistemas e Integração possui 10 questões dentro dos conhecimentos específicos. Dentro dela aparecem expressamente arquitetura cliente-servidor, sistemas web e dispositivos móveis, padrões arquiteturais MVC e DDD, microsserviços, integração de sistemas, APIs RESTful, JSON, webhooks, PDPJ, Spring, service discovery, API Gateway, persistência, mensageria, Docker, Kubernetes, Rancher e ambientes distribuídos/escaláveis.

O recorte desta aula é o núcleo inicial: arquitetura cliente-servidor, arquitetura em camadas e MVC. Esse núcleo será reaproveitado em REST, microsserviços, Spring, mensageria e cloud.

2.1 Por que este assunto é perigoso na FGV?

A FGV costuma cobrar leitura fina de conceito. O candidato experiente erra quando pensa que o assunto é simples, mas não percebe que a alternativa trocou o nível de análise.

Quatro níveis de análise aparecem com frequência:

- Comunicação: quem solicita e quem responde? Exemplo: cliente-servidor via HTTP.
- Organização lógica: onde fica cada responsabilidade? Exemplo: apresentação, negócio e dados.
- Padrão de apresentação: quem recebe entrada, manipula modelo e seleciona resposta? Exemplo: MVC.
- Distribuição física: em quantos processos, servidores ou nós roda? Exemplo: monolito, N-tier, cluster, microsserviços.

3 Visão macro: três conceitos que não são a mesma coisa

Pense em uma aplicação interna do Banco do Brasil ou do TJSC: um front-end Angular chama uma API Java/Quarkus; essa API aplica regras de negócio, consulta banco e devolve JSON. Essa mesma aplicação pode ser descrita por vários ângulos.

Cliente-servidor: o Angular, no navegador, envia requisição. O backend responde. Exemplo: o browser chama GET /processos/id e a API devolve JSON.

Camadas: o backend é organizado por responsabilidades. Exemplo: Resource, Service, Repository e Banco.

MVC: há separação entre entrada/controle, modelo e apresentação/visão. Exemplo: Controller/Resource recebe requisição; Model representa dados/regras; View pode ser tela HTML ou representação retornada.

Regra de ouro: cliente-servidor fala de interação entre partes; camadas falam de organização interna por responsabilidade; MVC fala de separação entre modelo, visualização e controle, principalmente na camada de apresentação/web.

Mapa mental textual: Usuário usa cliente Angular; o cliente envia requisição HTTP; o servidor/API em Quarkus ou Spring recebe a chamada; um Controller ou Resource delega para Application Service ou Use Case; a regra de negócio fica no domínio; Repository/DAO/ORM acessa o banco; a resposta retorna ao cliente como HTML, JSON, erro ou redirecionamento.

4 Arquitetura cliente-servidor

A arquitetura cliente-servidor é um modelo de computação distribuída no qual um cliente solicita recursos ou serviços a um servidor. No contexto web, a forma mais comum é o cliente iniciar uma requisição HTTP e o servidor responder com uma mensagem HTTP.

Segundo a MDN, HTTP é um protocolo cliente-servidor: requisições são iniciadas pelo destinatário do recurso, normalmente o navegador, e as mensagens do cliente são requests, enquanto as respostas do servidor são responses. HTTP também é stateless: uma requisição não mantém, por si só, vínculo de estado com outra.

4.1 Elementos principais

- Cliente: inicia requisição, apresenta interface ou consome serviço. Exemplo: Angular no navegador, app mobile, Postman, outro backend consumidor.
- Servidor: processa requisição, aplica lógica, acessa dados e devolve resposta. Exemplo: API Java/Quarkus, serviço Spring Boot, servidor de arquivos, servidor de autenticação.
- Protocolo: define regras de comunicação. Exemplo: HTTP/HTTPS.
- Recurso ou serviço: objeto ou funcionalidade exposta. Exemplo: /usuarios, /processos, /pagamentos.

4.2 Cliente não significa necessariamente tela

Em prova, cliente é qualquer componente que consome um serviço. Pode ser browser, aplicativo mobile, sistema legado, job batch ou outro microsserviço. Servidor é o componente que oferece o recurso ou serviço.

Pegadinha: em uma integração entre dois backends, um deles pode estar no papel de cliente e o outro no papel de servidor. O papel depende da interação, não do tipo de software.

4.3 Servidor lógico x servidor físico

Uma questão pode dizer que o servidor atende às requisições. Isso não significa que exista apenas uma máquina. O servidor pode ser um cluster, um conjunto de pods Kubernetes, instâncias atrás de load

balancer ou uma API Gateway encaminhando para serviços internos. Para o cliente, existe um endpoint lógico; por trás, pode haver várias instâncias.

Exemplos:

- Browser chama API REST: cliente-servidor, porque o cliente inicia request e o servidor responde.
- App mobile chama backend: cliente-servidor, pelo mesmo motivo.
- Backend A chama Backend B: A atua como cliente; B atua como servidor.
- API em cinco pods: continua cliente-servidor, pois a distribuição física não elimina o papel lógico.
- **Mensageria assíncrona**: depende do caso; fila/evento muda o padrão de interação e não é o cliente-servidor HTTP clássico.

4.4 Vantagens e desvantagens

Vantagens: centralização de regras e dados; reuso por múltiplos clientes; controle de segurança; possibilidade de escalar o servidor; facilidade de auditoria e monitoramento no backend.

Desvantagens: **dependência de rede; risco de gargalo no servidor; ponto único de falha quando não há redundância; acoplamento contratual entre cliente e API.**

5 Arquitetura em camadas

A arquitetura em camadas **organiza o sistema por responsabilidades**. Em vez de deixar interface, regra de negócio e acesso a dados misturados, ela cria separações lógicas. **A documentação da Microsoft destaca que, conforme aplicações crescem em complexidade, uma forma de gerenciá-las é quebrá-las por responsabilidades ou concerns, seguindo separação de preocupações.**

5.1 Camadas clássicas

- **Apresentação**: interação com usuário ou entrada HTTP. Exemplo: Angular no front; Resource/Controller no backend. Erro comum: colocar regra negocial pesada no controller.
- Aplicação: orquestra casos de uso, transações e fluxo de aplicação. Exemplo: ProcessoService, AprovarPagamentoUseCase.
- Domínio/Negócio: concentra regras centrais do negócio. Exemplo: entidades, objetos de valor, políticas e validações negociais.
- Persistência/Infraestrutura: acesso a dados e detalhes técnicos. Exemplo: Repository, Panache, JPA/Hibernate, client REST externo.
- Banco/serviços externos: armazenamento e dependências externas. Exemplo: Oracle/PostgreSQL, Redis, RabbitMQ, APIs corporativas.

5.2 Regra de dependência

No modelo em camadas tradicional, cada camada deve depender preferencialmente da camada imediatamente inferior ou de abstrações bem definidas. A UI não deve conversar diretamente com o banco. O controller não deve montar SQL manualmente e aplicar regra sensível. A camada de negócio não deve depender de detalhes visuais do Angular.

Frase FGV: **a arquitetura em camadas favorece separação de responsabilidades, manutenibilidade, testabilidade, reuso e redução do impacto de mudanças.**

5.3 Layer x tier

Essa é uma das principais pegadinhas. **Layer é separação lógica de responsabilidades.** **Tier é separação física de implantação.** Um sistema pode ter três camadas lógicas e rodar em um único processo; ou pode ter três tiers físicos, com front-end, backend e banco em servidores separados.

- Camada/layer: natureza lógica. Exemplo: Controller, Service, Repository.
- Tier: natureza física/deployment. Exemplo: servidor web, servidor de aplicação, servidor de banco.
- N-layer: organização lógica em N camadas. Pode rodar em um único servidor.
- N-tier: distribuição física em N níveis. Envolve separação em processos, máquinas ou ambientes.

5.4 O que a FGV pode cobrar

Camadas reduzem acoplamento, mas não eliminam complexidade. Separar camadas ajuda testes unitários e substituição de infraestrutura. Camadas mal implementadas podem virar arquitetura “lasanha”: muitas passagens mecânicas sem valor. Arquitetura em camadas não é sinônimo de microserviços. MVC pode estar dentro de uma arquitetura em camadas, mas não é a mesma coisa.

6 MVC - Model, View e Controller

MVC é um padrão que separa responsabilidades em três papéis: Model, View e Controller. Em aplicações web, essa separação ajuda a organizar entrada do usuário, regras/dados e apresentação. No Spring MVC, a documentação oficial descreve o uso do padrão Front Controller por meio do DispatcherServlet, que centraliza o processamento da requisição e delega o trabalho a componentes configuráveis.

6.1 Papéis do MVC

- Model: representa dados, estado e regras associadas ao domínio ou à aplicação. Não é apenas tabela de banco.
- View: apresenta informação ao usuário ou define representação de saída. Pode ser HTML, template, componente de front ou representação retornada.
- Controller: recebe entrada/requisição, coordena fluxo e seleciona resposta. Não deve concentrar toda a regra de negócio.

6.2 MVC em web clássico

Fluxo MVC clássico: browser envia request; o Front Controller/DispatcherServlet recebe; o Controller específico executa a coordenação; Model, Services ou Domain participam do processamento; o View Resolver escolhe a tela/template; o HTML retorna ao browser.

No Spring MVC clássico, o Controller pode retornar o nome de uma View, e o framework resolve o template. Esse modelo aparece em aplicações server-side rendered.

6.3 MVC em APIs REST

Em **APIs REST modernas, o controller frequentemente devolve JSON** em vez de escolher uma página HTML. Isso não apaga a ideia de separação, mas muda a forma da View. Para prova, o ponto seguro

é: controller não deve concentrar regra de negócio; ele recebe a requisição, valida aspectos de entrada, chama serviços/casos de uso e devolve resposta adequada.

Pegadinha: REST não é sinônimo de MVC. REST é estilo arquitetural de comunicação por recursos; MVC é padrão de separação de responsabilidades na aplicação, especialmente na camada web/apresentação.

6.4 Exemplo com Spring MVC

Exemplo Spring MVC: um RestController mapeado para /processos recebe a requisição GET /processos/id, extrai o identificador por PathVariable, chama ProcessoService.consultar(id) e devolve um ProcessoDTO. A regra de negócio não deve ficar concentrada no método do controller.

Nesse exemplo, o controller recebe a requisição HTTP, extrai o identificador, chama um serviço e devolve uma representação. A regra de negócio de consulta não deveria estar toda dentro do método do controller.

6.5 Exemplo com Quarkus

Exemplo Quarkus/JAX-RS: um ProcessoResource mapeado para /processos recebe uma chamada GET /id, obtém o parâmetro id, delega a consulta ao ProcessoService e retorna um DTO em JSON. O nome Resource muda, mas o papel arquitetural é semelhante ao de Controller.

Em Quarkus/JAX-RS, o nome comum é Resource, mas o papel arquitetural é semelhante ao Controller: receber requisição, delegar e retornar resposta.

7 Exemplo integrado: Angular + Java/Quarkus + banco

Imagine um sistema do TJSC para consulta de processos administrativos internos. O usuário acessa uma tela Angular, pesquisa pelo número do processo e visualiza metadados.

Fluxo: usuário preenche formulário Angular; Angular chama GET /api/processos/numero; ProcessoResource recebe a requisição; ProcessoService valida regra de acesso e orquestra consulta; ProcessoRepository executa consulta via ORM; banco retorna dados; API devolve JSON; Angular renderiza o resultado.

Classificação por conceito:

- Cliente-servidor: sim. O Angular, como cliente, inicia requisição; a API, como servidor, responde.
- Arquitetura em camadas: sim, se o backend separar entrada, aplicação/negócio e persistência.
- MVC: parcialmente/por analogia. Há View no Angular, Controller/Resource no backend e Model/DTO/entidades/serviços. Em APIs, MVC aparece de forma menos clássica que em páginas renderizadas no servidor.
- Microserviço: não necessariamente. Pode ser monolito modular em camadas. Microserviço depende de decomposição em serviços autônomos, deployment independente e fronteiras bem definidas.

Alerta de prova: se a alternativa disser que basta usar Controller, Service e Repository para haver microserviços, está errada. Isso indica camadas internas, não necessariamente arquitetura de microserviços.

8 Quadros comparativos para prova

8.1 MVC x arquitetura em camadas

- Foco do MVC: separar Model, View e Controller.
- Foco das camadas: separar responsabilidades lógicas do sistema.
- Escopo típico do MVC: camada web/apresentação.
- Escopo típico das camadas: sistema inteiro ou backend.
- Exemplo de MVC: Spring MVC com Controller e View resolver.
- Exemplo de camadas: Controller -> Service -> Repository -> Database.
- Pegadinha: MVC não é obrigatoriamente microserviço; camadas não exigem distribuição física.

8.2 Cliente-servidor x camadas x tiers

- Cliente-servidor responde: quem solicita e quem fornece serviço? Exemplo: navegador solicita dados à API.
- Camadas respondem: como organizar responsabilidades no código? Exemplo: Controller, Service, Repository.
- Tiers respondem: como distribuir fisicamente a solução? Exemplo: front em CDN, API em cluster, banco gerenciado.
- MVC responde: como separar entrada, modelo e apresentação? Exemplo: Controller manipula request e escolhe view/response.

8.3 Onde colocar cada responsabilidade

- Validar se usuário tem perfil para aprovar processo: Service/regra de aplicação, pois é regra negocial/de autorização.
- Formatar máscara de CPF na tela: View/front-end, pois é apresentação.
- Receber parâmetro HTTP: Controller/Resource, pois é entrada web.
- Executar consulta SQL ou ORM: Repository/Data Access, pois é persistência.
- Gerar DTO de resposta: Controller ou Application Service, conforme padrão do projeto; é mapeamento de contrato, não regra central de domínio.

9 Pegadinhas FGV

1. MVC é uma arquitetura em três camadas. Cuidado: MVC tem três papéis, **mas não é sinônimo de arquitetura em três camadas**.
2. Cliente-servidor exige servidor único. Errado: **o servidor pode ser lógico, composto por várias instâncias atrás de balanceador**.
3. Camada é o mesmo que tier. Errado: camada é lógica; tier é físico/deployment.
4. Controller deve conter a regra de negócio. Errado como regra geral: Controller coordena entrada e resposta; regra central fica em serviço/domínio.

5. View é sempre HTML server-side. Cuidado: em MVC clássico, frequentemente; em aplicações modernas, a apresentação pode estar no Angular e a API pode devolver JSON.
6. REST, MVC e camadas são concorrentes. Errado: REST pode ser usado em uma arquitetura cliente-servidor, implementada em backend em camadas.
7. Repository é camada de negócio. Cuidado: Repository abstrai persistência/acesso a dados.
8. Se usa Angular, não tem MVC. Depende: Angular não implementa MVC clássico de servidor, mas há separação de responsabilidades no front e integração com controller/resource no backend.

Resumo decorável:

Resumo decorável: Cliente-servidor = comunicação, cliente pede e servidor responde. Camadas = organização lógica, UI -> negócio -> dados. MVC = padrão de apresentação, Model + View + Controller. Tier = implantação física, máquinas/processos/ambientes.

10 Questões autorais comentadas - padrão TJSC/FGV

As questões abaixo são autorais e foram construídas para treinar distinção conceitual provável na banca. Não reproduzem questões reais.

Questão 1

Em uma aplicação web do Poder Judiciário, um navegador envia uma requisição HTTP para consultar dados de um processo, e uma API no servidor processa a solicitação e devolve uma resposta em JSON. Sob a perspectiva arquitetural, a descrição caracteriza principalmente:

- A) um padrão de persistência objeto-relacional.
- B) um modelo peer-to-peer, pois cliente e servidor sempre possuem as mesmas responsabilidades.
- C) uma arquitetura cliente-servidor, pois há solicitação iniciada pelo cliente e resposta provida pelo servidor.**
- D) um padrão de mensageria assíncrona, pois JSON pressupõe uso de fila.
- E) uma arquitetura exclusivamente em três camadas físicas.

Gabarito: C. A correta é C porque descreve o núcleo do modelo cliente-servidor: o cliente inicia a requisição e o servidor processa/devolve resposta. A está errada porque ORM/persistência não é o foco. B está errada porque peer-to-peer pressupõe simetria maior entre nós. D inventa relação obrigatória entre JSON e fila. E confunde cliente-servidor com implantação física em tiers.

Questão 2

No padrão MVC aplicado a uma aplicação web, o componente responsável por receber a entrada do usuário ou a requisição, coordenar o fluxo e acionar os componentes necessários para produzir a resposta é o:

- A) Controller.**
- B) Repository.
- C) View.
- D) Database.
- E) Message Broker.

Gabarito: A. O Controller recebe a entrada/requisição e coordena o fluxo. Repository pertence à persistência, não ao MVC clássico. View apresenta o resultado. Database armazena dados. Message Broker intermedia mensagens/eventos e pertence ao tema de mensageria, não ao núcleo MVC.

Questão 3

Considere uma aplicação organizada em apresentação, lógica de negócio e acesso a dados, mas implantada em um único servidor de aplicação. A respeito dessa solução, assinale a afirmativa correta.

- A) Não há arquitetura em camadas, pois camadas exigem servidores físicos distintos.
- B) Há necessariamente três microserviços, um para cada responsabilidade.

- C) O uso de camadas elimina a necessidade de testes.
- D) A organização descrita é incompatível com aplicações monolíticas.
- E)** Há separação lógica em camadas, ainda que não haja separação física em tiers.

Gabarito: E. A correta é E. Camadas são separações lógicas; tiers são separações físicas. A confunde layer com tier. B confunde camadas internas com microsserviços. C é absurda: camadas facilitam testes, não os eliminam. D erra porque monolitos podem ser bem organizados em camadas.

Questão 4

Em uma API Java, a classe `ProcessoResource` recebe a requisição HTTP, chama `ProcessoService`, que aplica regras de negócio, e este utiliza `ProcessoRepository` para acessar o banco de dados. Essa organização ilustra principalmente o princípio de:

- A) replicação síncrona de banco de dados.
- B)** separação de responsabilidades em camadas.
- C) roteamento peer-to-peer entre clientes.
- D) compressão de payload HTTP.
- E) auditoria temporal de entidades.

Gabarito: B. A sequência `Resource -> Service -> Repository` evidencia camadas/responsabilidades. A trata de banco. C trata de modelo distribuído diverso. D é preocupação de protocolo/performance. E lembra Envers/auditoria, assunto de outra aula.

Questão 5

No Spring MVC, o `DispatcherServlet` é frequentemente associado ao padrão Front Controller. Sua função central é:

- A) substituir o banco de dados relacional.
- B) executar exclusivamente regras de negócio de domínio.
- C) converter uma aplicação monolítica automaticamente em microsserviços.
- D) centralizar o processamento inicial das requisições e delegar o trabalho a componentes apropriados.
- E) garantir que toda requisição HTTP seja stateful.

Gabarito: D. O `DispatcherServlet` centraliza a entrada das requisições no Spring MVC e delega para handlers/controllers e demais componentes. A, B e C atribuem funções que ele não possui. E contradiz a característica geral do HTTP, que é stateless por natureza, embora sessões possam ser construídas sobre ele.

Questão 6

Sobre MVC e arquitetura em camadas, assinale a afirmativa mais adequada.

- A) MVC e arquitetura em camadas são sinônimos perfeitos.

- B) MVC é uma técnica exclusiva de banco de dados.
- ☒ C) MVC pode coexistir com arquitetura em camadas, mas trata de uma separação conceitual distinta.
- D) Arquitetura em camadas impede o uso de REST.
- E) O uso de Controller elimina a necessidade de camada de negócio.

Gabarito: C. A correta é C. MVC separa Model, View e Controller; camadas organizam responsabilidades mais amplas, como apresentação, negócio e persistência. A ignora a diferença. B não tem relação. D é falsa: REST pode ser implementado em sistemas em camadas. E é uma armadilha comum: Controller não deve concentrar tudo.

Questão 7

Uma API do TJSC é acessada por um aplicativo mobile. Para aumentar disponibilidade, a API roda em várias instâncias atrás de um balanceador de carga. Considerando o modelo de interação entre app e API, é correto afirmar que:

- A) o modelo cliente-servidor permanece aplicável, pois o servidor pode ser visto como um serviço lógico composto por várias instâncias.
- B) o modelo cliente-servidor deixa de existir quando há balanceador de carga.
- C) a aplicação passa automaticamente a ser peer-to-peer.
- D) o aplicativo mobile passa a ser o servidor, pois possui interface gráfica.
- E) a existência de múltiplas instâncias impede o uso de HTTP.

Gabarito: A. A correta é A. A multiplicidade de instâncias não elimina o papel lógico de servidor. B e C confundem distribuição física com modelo de interação. D erra porque interface gráfica não define o papel de servidor. E é falsa: HTTP é amplamente usado com balanceadores e múltiplas instâncias.

Questão 8

Em uma aplicação corporativa, a tela front-end acessa diretamente o banco de dados de produção para montar consultas, sem passar pela API ou pela camada de negócio. Sob a ótica da arquitetura em camadas, essa decisão é problemática porque:

- A) impede que o banco armazene dados relacionais.
- B) obriga o uso de MVC clássico server-side.
- C) transforma automaticamente o sistema em event-driven.
- D) elimina a necessidade de autenticação.
- E) viola a separação de responsabilidades e acopla apresentação à persistência.

Gabarito: E. A correta é E. A UI acessar diretamente o banco mistura responsabilidades, aumenta acoplamento, dificulta segurança, auditoria e evolução. A não tem relação. B é falsa: o problema não é ausência de MVC clássico. C e D são conclusões sem base técnica.

Questão 9

Em uma API REST, um método de Controller contém validações de regra de negócio complexas, cálculo de prazos, decisão de alçada e consultas SQL manuais. Considerando boas práticas de separação em camadas, a crítica mais adequada é que:

- A) Controllers devem sempre acessar banco diretamente para reduzir classes.
- B) o Controller está acumulando responsabilidades que deveriam ser separadas em serviços/domínio e persistência.
- C) a aplicação deixou de ser cliente-servidor por possuir regras de negócio.
- D) todo Controller deve ser substituído por Message Broker.
- E) camadas são incompatíveis com APIs REST.

Gabarito: B. A correta é B. Controller deve ser fino o suficiente para coordenar entrada e saída, delegando regra de negócio para serviço/domínio e acesso a dados para repositório/infra. A é antipadrão. C não faz sentido. D confunde integração assíncrona com entrada HTTP. E é falsa.

Questão 10

Um tribunal deseja disponibilizar a mesma funcionalidade de consulta para uma aplicação web Angular e para um aplicativo móvel. A alternativa mais aderente a uma solução manutenível, considerando os conceitos desta aula, é:

- A) duplicar toda a regra de negócio em cada cliente, evitando backend.
- B) permitir que cada cliente acesse diretamente o banco de dados.
- C) colocar todas as regras no componente visual para reduzir chamadas HTTP.
- D) expor uma API no servidor, organizada em camadas, para ser consumida pelos diferentes clientes.
- E) usar MVC como substituto obrigatório de banco de dados.

Gabarito: D. A correta é D. Uma API centralizada no servidor, organizada em camadas, permite reuso por web e mobile, preservando regras e segurança no backend. A duplica regra. B viola camadas e segurança. C mistura apresentação e negócio. E é tecnicamente sem sentido.

10.1 Distribuição do gabarito

A: questões 2 e 7. B: questões 4 e 9. C: questões 1 e 6. D: questões 5 e 10. E: questões 3 e 8. Distribuição equilibrada: duas respostas por alternativa.

11 Flashcards finais

Use estes cartões para revisão ativa.

1. O que define arquitetura cliente-servidor? Cliente solicita recursos/serviços; servidor processa e responde.
2. Cliente sempre é navegador? Não. Pode ser app mobile, outro backend, Postman, job ou sistema externo.

3. Servidor precisa ser uma única máquina? Não. Pode ser endpoint lógico com várias instâncias.
4. HTTP é stateful? Não. HTTP é stateless; sessões podem ser construídas com cookies/tokens.
5. O que é arquitetura em camadas? Organização lógica por responsabilidades.
6. Camada e tier são sinônimos? Não. Camada é lógica; tier é físico/deployment.
7. Quais são camadas comuns? Apresentação, aplicação/negócio, persistência/infraestrutura e dados.
8. UI deve acessar banco diretamente? Como regra, não. Viola separação de responsabilidades.
9. O que é MVC? Padrão que separa Model, View e Controller.
10. Função do Controller? Receber entrada/requisição, coordenar fluxo e acionar componentes.
11. Função da View? Apresentar/representar a saída ao usuário ou cliente.
12. Função do Model? Representar dados, estado e regras do domínio/aplicação.
13. MVC é igual a três camadas? Não. MVC tem três papéis; camadas são organização lógica ampla.
14. REST é MVC? Não. REST é estilo de comunicação por recursos; MVC é padrão de organização.
15. Controller deve concentrar regra de negócio? Não. Regra central deve ir para service/domínio.
16. Repository pertence a qual preocupação? Persistência/acesso a dados.
17. Uma aplicação em camadas precisa ser microserviços? Não. Pode ser monolito modular.
18. Angular + API Java pode ser cliente-servidor? Sim. Angular é cliente; API é servidor.
19. O que o DispatcherServlet faz no Spring MVC? Atua como Front Controller central, recebendo requisições e delegando processamento.
20. Regra de ouro da aula? Cliente-servidor = comunicação; camadas = organização lógica; MVC = Model/View/Controller.

12 Revisão ativa pós-aula

Antes de ir para o Qconcursos, faça esta revisão de cinco minutos:

1. Diga em voz alta: cliente-servidor é comunicação; camadas são organização lógica; MVC é Model/View/Controller.
2. Resolva cinco questões filtradas por FGV + Arquitetura de Software.
3. Para cada erro, marque se foi confusão de nível: comunicação, organização lógica, apresentação ou implantação física.
4. Se errar MVC, revise papéis de Model, View e Controller.
5. Se errar camadas, revise layer x tier.

13 Referências técnicas consultadas

- FGV Conhecimento - Concurso Público TJSC 2026: <https://conhecimento.fgv.br/concursos/tjscservidor26>
- MDN Web Docs - Overview of HTTP: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>
- Spring Framework Documentation - DispatcherServlet: <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet.html>
- Microsoft Learn - Common web application architectures: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>
- Microsoft Learn - Architectural principles: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/architectural-principles>

Material autoral para estudo. Não substitui a leitura do edital nem a resolução de questões reais da banca.